

Spec S13 · Gestión de usuarios con baja + Import histórico (Excel 2026)

```
---
title: "S13 · Usuarios con desactivación e importación del histórico
Excel"
status: "approved"
version: "1.0"
priority: "alta"
estimated_effort: "2-3 días"
dependencies: ["DB dev con historia 2026 cargada (2215
atenciones)", "docs/import-historico-decisiones.md (cerrado)"]
---
```

Historias de usuario

Como jefe de soporte, **quiero** dar de baja a técnicos retirados sin borrar sus atenciones **para** que la historia quede atribuida a nombre propio y no aparezcan como equipo activo.

Como jefe de soporte, **quiero** importar las 556 atenciones 2026 de los 2 técnicos retirados desde sus planillas Excel **para** que las estadísticas del año estén completas.

Objetivo

Cerrar el año 2026 completo en dev: 2 usuarios inactivos + sus 556 atenciones importadas, visibles en stats y listas, sin romper los 99 tests existentes.

Decisiones heredadas (cerradas · no reabrir)

Todo lo de docs/import-historico-decisiones.md es vinculante: 557 filas · 556 (se descarta Gabriel ID 363), 46 mapeos origen·canónico (Grupo A tipos 64 filas, Grupo B negocio 107 filas), 3 áreas nuevas directas de Extras, 4 normalizaciones, 3 correcciones de fila, 6 grupos de duplicados con veredicto, columnas ya válidas tras normalizar Correo · Interno Sin columna de procedencia (declinada por el usuario). Sin pre-2026, sin tocar prod (S12 descartada).

Stack Tecnológico (OBLIGATORIO)

| Capa | Tecnología | Versión | Notas | | Backend | FastAPI + SQLAlchemy sync + psycopg | repo actual | snake_case, DbSession, errores vía app.core.errors | | Migraciones | alembic | repo actual | nueva revisión encadenada al head vigente | alembic heads antes de crear | | Frontend | Django templates server-rendered | repo actual | consume FastAPI vía FASTAPI_URL, helpers | | Base de datos | PostgreSQL | dev + _test | tests exigen nombre terminado en _test (backend-fastapi/tests/conftest.py) | | Auth | JWT Bearer X-User-Id compatible .NET | actual | require_user + is_privileged en app/core/security.py | | Scripts | Python stdlib (csv, datetime) | · | Sin librerías nuevas. Reusar construir_resolutor() y seed_atenciones() | | Tests | pytest (68 backend) + Django (31 en tests/test_views.py) | actual | Todos deben seguir verdes |

Arquitectura

Diagrama de componentes (textual)

```
[Django: /usuarios/] --HTTP--> [FastAPI: /api/usuarios] -->
[Servicio: Usuarios] --> [DB: Usuarios.Activo]
[Django: /jerarquia/] --HTTP--> [FastAPI: /api/jerarquia] --> [DB:
Areas] (patrón a espejar, sin cambios)
[Script: importar_historico.py] --> [mapeo-areas.csv +
construir_resolutor()] --> [atenciones_historico.csv] -->
[seed_atenciones()] --> [DB]
[Django: Inicio "El equipo ahora"] <-- GET /api/usuarios (solo
activos)
[Django: Dashboard/stats] <-- GET /api/atenciones/stats (SIN filtro
Activo · invariante INV-01)
```

Mapa de archivos

```
# Archivos a crear:
backend-fastapi/alembic/versions/XXXX_activo_usuarios.py # ADD
COLUMN Activo + backfill, downgrade DROP COLUMN
backend-fastapi/scripts/importar_historico.py # lee 2
CSV Excel, normaliza, resuelve, emite atenciones_historico.csv
atenciones_historico.csv # (raíz
repo) 556 filas en formato COLUMNAS de seed; artefacto regenerable
frontend-django/templates/atenciones/usuarios.html #
pantalla /usuarios/ espejando jerarquia.html
docs/spec-s13-usuarios-e-import-historico.md # esta
spec
# Archivos a modificar:
backend-fastapi/app/models/usuario.py # + campo activo
(columna "Activo", Boolean, default True)
backend-fastapi/app/schemas/usuario.py # + activo en
UsuarioOut; + UsuarioCreateIn / ActivoIn
backend-fastapi/app/routers/usuarios.py # + POST "", + PATCH
/{id}/activo, + filtro ?incluir_inactivos en GET ""
backend-fastapi/app/routers/auth.py # login rechaza
inactivos (403); sin cambio de firma
backend-fastapi/app/core/security.py # require_user rechaza
inactivos (401)
mapeo-areas.csv # + 46 filas
(area_actual · nuevo_nombre); ver § Contratos
mapeo-areas-dedup.csv # + 3 filas (áreas nuevas
en Extras)
frontend-django/atenciones/urls.py # + path("usuarios/") +
path("usuarios/accion/")
frontend-django/atenciones/views.py # + usuarios_vista +
usuarios_accion_vista (espejo jerarquía)
frontend-django/templates/atenciones/_navbar.html # + entrada
Usuarios (solo can_dashboard, sección Equipo)
# Archivos a NO tocar:
backend-fastapi/app/routers/atenciones.py # _nombres() y
get_stats() NO filtran por Activo (INV-01)
```

```
backend-fastapi/scripts/seed.py          # reusar
seed_atenciones() sin cambios
backend-fastapi/scripts/extraer_septiembre.py # reusar
construir_resolutor() sin cambios
frontend-django/tests/test_views.py      # solo agregar tests, no
cambiar existentes
```

Contratos de Datos

API · Request / Response

```
# POST /api/usuarios (Jefe; 201)
class UsuarioCreateIn(BaseModel):
    email: str          # único, case-insensitive; se guarda
    lower().strip()
    display_name: str   # min 1, max 255
    role: str           # "Tecnico" | "Jefe" | "Auxiliar"
    password: str | None = None # None · PasswordHash NULL (sin
    login). Si viene: min 8 chars
    activo: bool = True
# Response 201: UsuarioOut del creado.
# PATCH /api/usuarios/{id}/activo (Jefe; 200)
class ActivoIn(BaseModel):
    activo: bool
# Response 200: UsuarioOut actualizado.
# GET /api/usuarios?incluir_inactivos=false (auth; 200)
# Por defecto SOLO Activo=true (alimenta "El equipo ahora" en
    Inicio).
# Con incluir_inactivos=true devuelve todos (pantalla /usuarios/).
# UsuarioOut suma:
class UsuarioOut(BaseModel):
    # ...campos actuales...
    activo: bool = True
```

Auth / login:

```
# POST /api/auth/login con usuario inactivo · 403 {"detail": "Usuario
    desactivado"}
# require_user con usuario inactivo (baja a mitad de sesión) · 401
# PasswordHash NULL + intento login · ya existe 401 "sin contraseña
    asignada" (se mantiene)
```

Modelo de Base de Datos

```
-- Migración alembic (aditiva, reversible):
ALTER TABLE "Usuarios" ADD COLUMN "Activo" BOOLEAN NOT NULL DEFAULT
    TRUE;
-- downgrade: ALTER TABLE "Usuarios" DROP COLUMN "Activo";
-- Backfill implícito por DEFAULT: los 10 usuarios existentes quedan
    Activo=true.
-- Índice: no (baja cardinalidad, tabla de ~12 filas).
```

```
# app/models/usuario.py
activo: Mapped[bool] = mapped_column("Activo", Boolean,
nullable=False, default=True)
```

CSV · 46 mapeos + 3 áreas nuevas

mapeo-areas.csv (columnas area_actual, grupo_padre, grupo, nuevo_nombre, activo, notas): append 46 filas con activo=1, grupo_padre/grupo vacíos salvo que el destino canónico los tenga (solo se mapea texto·nombre; la jerarquía la resuelve el catálogo). Conteo a verificar en implementación: Grupo A 21 claves · 64 filas de origen; Grupo B 25 claves · 107 filas. Ver § Detalle de verificación.

mapeo-areas-dedup.csv (columnas grupo_padre, grupo, nombre, activo): append 3 filas:

```
Extras,,Colegio Domingo Savio,1
Extras,,Constructora,1
Extras,,Cardio Salud,1
```

Script importador · formato de salida

importar_historico.py emite EXACTAMENTE las COLUMNAS de extraer_septiembre.py (las que seed_atenciones() espera):

```
tecnico_email, area, colaborador_email, medio_solicitud,
usuario_solicitante,
categoria, descripcion, solucion, observaciones, enlace_apoyo,
fuera_de_turno, fecha_registro, created_at
```

Mapeo fijo por columna:

| Columna destino | Origen / valor | ---|---| | tecnico_email | dueño del archivo: Gabriel Gabriel.Torrico@upds.edu.bo, Deymar Deymar.Lozano@upds.edu.bo || area | área origen · corrección de fila (3) construir_resolutor() (clave exacta, fallback normalizado). Sin resolver · abortar (ver ERR-S) || colaborador_email | "" siempre (el origen no tiene colaborador) || medio_solicitud | origen, con Correo · Interno · Debe · {Presencial, Interno, WhatsApp, E-ticket} o abortar || usuario_solicitante | origen tal cual. Debe · {ADM, BEC, DOC, EST} o abortar || categoria | origen tal cual. Debe · 8 válidas (CATEGORIAS_VALIDAS) o abortar || descripcion / solucion | texto tal cual (strip) || observaciones / enlace_apoyo | "" si vacío N/A (case-insensitive, con variantes n/a, N/a) o strip·vacío || fuera_de_turno | "false" siempre (el origen no trae hora; ver RN-11) || fecha_registro | DD/MM/YYYY · ISO, con las 4 normalizaciones aplicadas antes de parsear || created_at | derivación determinista § Contrato created_at |

Filtrado: descartar filas con ID vacío (relleno del export, ~2700 líneas) y Gabriel ID 363.

Contrato created_at (determinista, a prueba de colisiones, re-ejecutable)

El origen no trae hora. Regla:

```
created_at(row) = fecha_registro a las 12:00 America/La_Paz (=
16:00 UTC)
+ k segundos,
donde k = ordinal (0-based) del row dentro de su grupo
(fecha_registro),
ordenado por (tecnico_email, area, descripcion, solucion).
Formato ISO con offset: "2026-03-06T16:00:07+00:00".
```

Propiedades verificadas: máx 16 filas/día en el origen (05/01 Deymar) · offsets de segundos sobran;

`datetime.fromisoformat` lo parsea (formato que `seed_atenciones` ya usa); re-ejecutar produce valores idénticos
`seed_atenciones()` los salta (creado in ya) · segunda corrida inserta 0.

Guardia anti-colisión (el importador la ejecuta antes de insertar):

- 1 Candidatos = los 556 `created_at` derivados.
- 2 `existentes = SELECT created_at, usuario_id, fecha_registro FROM Atenciones.`
- 3 Si candidato · `existentes` `Y(usuario_id · {Gabriel, Deymar} O fecha_registro · la del candidato)` · abortar listando los choques (falla ruidoso, como `seed_atenciones` con áreas).
- 4 Si candidato · `existentes` con dueño y fecha coincidentes · skip (idempotencia, corrida repetida).
- 5 Caso contrario · insertar.

Frontend · /usuarios/ (espejo /jerarquia)

- `usuarios_vista`: GET, solo `_puede_dashboard` (si no · `redirect` a `atenciones_lista`, mismo patrón). Llama GET `/api/usuarios?incluir_inactivos=true`, render `usuarios.html` con `{usuarios, detalle, flash}`.
- `usuarios_accion_vista`: POST con `accion · {crear, activar, desactivar}` volver + flash en sesión, mismo esqueleto que `jerarquia_accion_vista`. Crear · POST `/api/usuarios`; activar/desactivar · PATCH `/api/usuarios/{id}/activo`.
- Navbar: entrada "Usuarios" (icono `mdi:account-cog-outline`) en sección Equipo dentro del bloque `{% if can_dashboard %}`, después de Horarios.

Flujo Principal

F1 · Crear usuario (Jefe)

- 1 Jefe abre `/usuarios/` · ve lista (activos + inactivos con badge).
- 2 Completa email + nombre + rol (+ password opcional) · POST `/usuarios/accion/` · POST `/api/usuarios`.
- 3 Sistema valida (email único, rol válido, password ·8 si viene) · 201 "Creado." · `redirect` con `volver`.

F2 · Desactivar / reactivar (Jefe)

- 1 Jefe elige usuario · "Desactivar" · PATCH `/api/usuarios/{id}/activo {"activo": false}`.
- 2 Desde ese momento: no aparece en GET `/api/usuarios` por defecto ni en "El equipo ahora"; su JWT vigente deja de validar (401); sus atenciones siguen en listas y stats (INV-01).

F3 · Import histórico (operador, una vez + re-corridas)

- 1 Verificar catálogo: 3 áreas nuevas existen en DB `seed_catálogo` ya corrido tras editar `mapeo-areas-dedup.csv`; verificar 46 claves sin colisión (§ verificación).
- 2 Crear 2 usuarios vía F1 con `activo=false`, sin password.
- 3 Correr `python scripts/importar_historico.py` · genera `atenciones_historico.csv` (556 filas) + reporte (filas por archivo, descartes, normalizaciones aplicadas).
- 4 Cargar vía `seed_atenciones()` contra ese CSV (`ATENCIONES_CSV=atenciones_historico.csv python scripts/actualizar_atenciones.py` o equivalente sin re-hashear passwords) · guardia anti-colisión · insert.

- 5 Re-correr · 0 nuevas (idempotencia).

Flujo Alternativo · A1: reactivar para corregir

- 1 Jefe reactiva (`activo=true`), el técnico entra y edita SOLO sus atenciones (regla de dueño en PUT/DELETE sin cambios), Jefe vuelve a desactivar.

Flujo Alternativo · A2: área nueva aparece a mitad del import

- 1 El importador aborta con `area sin resolver: 'X'` · operador agrega el mapeo a `mapeo-areas.csv` (o el área al catálogo) · re-corre (idempotente, no duplica lo ya cargado).

Flujo de Error · API

| Condición | Respuesta | HTTP | Mensaje | ---|---|---|---| Email duplicado (case-insensitive) | Conflict | 409 | "Email ya registrado" || Rol inválido | Validation Error | 400 | "Rol inválido. Use: Tecnico, Jefe, Auxilia'" || Password < 8 chars | Validation Error | 400 | "La contraseña necesita al menos 8 caracteres" || Jefe se desactiva a sí mismo | Validation Error | 400 | "No puedes desactivar tu propio usuario" || Usuario inexistente | Not Found | 404 | "Usuario no encontrado" || No-Jefe crea/desactiva | Forbidden | 403 | "Solo Jefe puede gestionar usuarios" || Login de inactivo | Forbidden | 403 | "Usuario desactivado" || Token de usuario dado de baja | Unauthorized | 401 | "Usuario desactivado" || Asignar colaborador inactivo en PUT /atenciones o batch | Validation Error | 400 | "ColaboradorId {id} está desactivado"

Catálogo de Errores

| Código | HTTP / Exit | Condición | Mensaje visible | Log | ---|---|---|---| ERR-U-001 | 409 | Email duplicado en POST /api/usuarios | "Email ya registrado" | warn || ERR-U-002 | 400 | Rol inválido | "Rol inválido" | warn || ERR-U-003 | 400 | Password < 8 | "La contraseña necesita" | warn || ERR-U-004 | 400 | Auto-desactivación | "No puedes desactivar tu propio usuario" | warn || ERR-U-005 | 404 | Usuario inexistente (PATCH activo) | "Usuario no encontrado" | info || ERR-U-006 | 403 | No-Jefe gestiona usuarios | "Solo Jefe puede gestionar usuarios" | warn || ERR-U-007 | 403 | Login inactivo | "Usuario desactivado" | info || ERR-U-008 | 401 | Token de inactivo en require_user | "Usuario desactivado" | info || ERR-U-009 | 400 | Colaborador inactivo asignado | "Colaborador · está desactivado" | warn || ERR-S-001 | exit 1 | Área sin resolver en importador | `area sin resolver: 'X'` (stderr, lista completa) | error || ERR-S-002 | exit 1 | Medio/solicitante/categoría fuera de conjunto cerrado | `medio invalido / solicitante invalido / categoria invalida: 'X'` | error || ERR-S-003 | exit 1 | Fecha imposible tras normalizaciones | `fecha invalida: archivo=X id=Y valor='Z'` | error || ERR-S-004 | exit 1 | Colisión `created_at` con fila ajena | `created_at colisiona: <iso> (fila archivo=X id=Y)` | error || ERR-S-005 | exit 1 | Técnico dueño no existe en Usuarios | `tecnico '.' no existe en Usuarios` (heredado de `seed_atenciones`) | error || · | exit 0 | Fila con ID vacío (relleno) | se salta, se cuenta en `omitidas_vacias` | info |

Reglas de Negocio

- 1 **RN-01:** Desactivar · borrar. Las atenciones conservan `usuario_id` y nombre propio.
 - 2 **INV-01 (invariante crítica):** GET /api/atenciones, `_nombres()` y `get_stats` NO filtran por Activo. Verificado en código: `hacerselect(Usuario).where(id in · )` sin filtro de rol ni estado. No agregar ninguno.
 - 3 **RN-02:** GET /api/usuarios por defecto solo activos (con `role IN ('Tecnico', 'Jefe')` como hoy); ?`incluir_inactivos=true` para la pantalla admin.
 - 4 **RN-03:** Sin login para inactivos (403) y sin sesión vigente (401 en `require_user`).
-

- PasswordHash NULL además impide login por contraseña (401 existente).
- 5 **RN-04:** Un Jefe no puede desactivarse a sí mismo (400).
 - 6 **RN-05:** No se puede asignar como colaborador (nuevo) a un usuario inactivo (400). Las colaboraciones históricas existentes no se tocan.
 - 7 **RN-06:** Crear usuario y activar/desactivar es solo-Jefe (is_privileged), igual que especialidades y jerarquía.
 - 8 **RN-07:** Migración aditiva Activo NOT NULL DEFAULT TRUE; los 10 existentes quedan activos; downgrade = DROP COLUMN.
 - 9 **RN-08:** created_at derivado es la clave de idempotencia del import (determinista, § Contrato). Nunca now() en el importador.
 - 10 **RN-09:** El importador nunca escribe credenciales ni toca Usuarios (los 2 usuarios se crean por F1/API, no por script).
 - 11 **RN-10:** fuera_de_turno=false para las 556 (el origen no trae hora; no inventar turnos).
 - 12 **RN-11:** Email único case-insensitive en creación (unc.lower como en login).
 - 13 **RN-12:** No hardcodear IDs de usuario en el importador; resolver por email (como seed_atenciones).

Detalle de verificación (implementador, antes de importar)

```
# 1. Ninguna de las 46 claves nuevas pisa una existente con otro
destino:
python3 -c "
import csv
base={r['area_actual'].strip():r['nuevo_nombre'].strip() for r in
csv.DictReader(open('mapeo-areas.csv',encoding='utf-8-sig'))}
# NUEVAS = dict con las 46 (origen->destino) de decisiones §Grupo A
+ §Grupo B
choques={k:(base[k],v) for k,v in NUEVAS.items() if k in base and
base[k]!=v}
assert not choques, choques
print('ok: 46 claves sin colision')
# 2. Resolver cubre el 100% de áreas origen de ambos CSVs (latin-1,
';', filtrar ID vacío).
# 3. alembic heads · una sola cabeza; la nueva revisión la encadena.
```

Criterios de Aceptación (Gherkin)

```
Feature: S13 usuarios con baja e import histórico
  Background:
    Given DB dev con 2215 atenciones y 10 usuarios activos
  Scenario: Crear y desactivar técnico retirado
    Given soy Jefe
    When POST /api/usuarios {email:"Gabriel.Torrico@upds.edu.bo",
display_name:"Gabriel Torrico", role:"Tecnico"}
    Then status 201 y Activo=true, PasswordHash NULL
    When PATCH /api/usuarios/{id}/activo {activo:false}
    Then GET /api/usuarios no lo incluye; GET
/api/usuarios?incluir_inactivos=true sí
  Scenario: Inactivo no entra ni opera
    Given usuario desactivado con JWT previo
    When POST /api/auth/login con sus credenciales
    Then 403 "Usuario desactivado"
```

```

    When GET /api/usuarios/me con su JWT previo
    Then 401
  Scenario: Atenciones de inactivos visibles (INV-01)
    Given Gabriel y Deymar inactivos con 556 atenciones
    When GET /api/atenciones/stats
    Then por_tecnico incluye "Gabriel Torrico" con 363 y "Deymar
Lozano" con 193
    And GET /api/atenciones?usuario_id={gabriel} devuelve sus filas
con usuario_nombre correcto
  Scenario: Import idempotente
    When corro el importador dos veces
    Then la segunda inserta 0 y el total de Gabriel+Deymar sigue en
556
  Scenario: GUI /usuarios/
    Given soy Jefe
    When abro /usuarios/
    Then veo altas, bajas con badge y flash de confirmación tras
cada acción
    Given no soy Jefe
    When abro /usuarios/
    Then redirect a /atenciones/

```

Mockups ASCII

```

+-----+-----+-----+-----+
| Usuarios                                [+ Nuevo usuario] |
+-----+-----+-----+-----+
| Nombre          | Rol      | Estado   | Acción  |
| Mattias Ribera R. | Tecnico  | activo   | [Baja]  |
| Gabriel Torrico   | Tecnico  | INACTIVO | [Alta]  |
| Deymar Lozano     | Tecnico  | INACTIVO | [Alta]  |
+-----+-----+-----+-----+
| Nuevo: [email____] [nombre____] [rol v] [Crear]      |
+-----+-----+-----+-----+

```

Definición de Estados de UI (/usuarios/)

Estado	Condición	Render	----- ----- -----	Loading	· (server-render, N/A) N/A · render síncrono como
jerarquía	Empty	Sin usuarios (imposible en práctica)	"Sin usuarios registrados"	Error	ApiError del backend
flash rojo + lista anterior intacta (patrón jerarquía)	Success	GET ok	tabla + badges + flash "		
Creado./Desactivado./Activado."					

Definition of Done (DoD)

```

# 1. Migración aplicada en dev y _test:
alembic upgrade head && psql $DATABASE_URL -c '\d "Usuarios"' #
columna "Activo" boolean not null default true
# 2. Los 10 existentes siguen activos:
psql $DATABASE_URL -c 'SELECT count(*) FROM "Usuarios" WHERE

```

```

"Activo"]=false;' # · 0 (antes del import)
# 3. API usuarios:
curl $API/api/usuarios -H "Authorization: Bearer $JEFE"
# · sin Gabriel/Deymar
curl "$API/api/usuarios?incluir_inactivos=true" ...
# · 12 usuarios, 2 con activo=false
curl -X POST $API/api/auth/login -d
'{"email":"Gabriel.Torrico@upds.edu.bo",...}' # · 403
# 4. Import (556 = 363 Gabriel + 193 Deymar):
psql $DATABASE_URL -c 'SELECT u."DisplayName", count(*) FROM
"Atenciones" a JOIN "Usuarios" u ON u."Id"=a."UsuarioId" WHERE
u."Email" IN (...) GROUP BY 1;'
# · Gabriel Torrico 363, Deymar Lozano 193
psql $DATABASE_URL -c "SELECT
to_char(\"FechaRegistro\", 'YYYY-MM'),count(*) FROM \"Atenciones\"
WHERE ... GROUP BY 1 ORDER BY 1;"
# · solo meses 2026-01..2026-04, total 556
# Re-corrida del importador · "atenciones nuevas: 0"
# 5. INV-01 · stats incluyen inactivos:
curl "$API/api/atenciones/stats" ... | jq '.por_tecnico[] |
select(.display_name=="Gabriel Torrico")' # · total 363
# 6. Tests:
cd backend-fastapi && DATABASE_URL=..._test pytest -q
# · 67 passed + 1 fallo PRE-EXISTENTE fuera de alcance:
# tests/test_realtime.py::test_connect_flip_y_disconnect
(acoplado al reloj:
# pasa solo fuera del turno 11:00-19:00 local de Diego; dentro de
ese horario falla).
# Fuera de ese horario · 68 passed.
cd frontend-django && set -a && source .env && set +a &&
.venv/bin/python manage.py test tests
# · 31 tests OK (+ los nuevos de /usuarios/). pytest NO existe en
frontend-django;
# el runner es el propio Django y requiere DJANGO_SECRET_KEY
desde .env.
# 7. Calidad: ruff/lsp sin errores en archivos tocados; sin
librerías nuevas en requirements.

```

Orden de Implementación

- Paso 1 · Datos** · editar mapeo-areas.csv (+46) y mapeo-areas-dedup.csv (+3) · correr \$seed_catálogo (vía seed/actualizar) · 3 áreas en DB.
 - Paso 2 · Migración + modelo + schemas** · migrar Active, Usuario. activo, UsuarioOut. activo, UsuarioCreateIn, ActivoIn.
 - Paso 3 · API** · POST "", PATCH /{id}/activo, filtro GET "", guards en auth. login + require_user, validación colaborador inactivo en atenciones.py (solo check, sin cambiar reads).
 - Paso 4 · Importador** · importar_historico.py · CSV · carga · DoD #4.
 - Paso 5 · Django** · urls + vistas + template + navbar.
 - Paso 6 · Tests** · backend (crear/desactivar/login-bloqueado/filtro default/INV-01 stats) + Django (/usuarios/ Jefe vs no-Jefe) · DoD #6.
-

Lo que NO está en alcance (explícito)

- · Columna de procedencia importado-vs-registrado (declinada por el usuario).
- · Historia pre-2026 ni tocar `proceso_Tecnico-python`, S12 descartada).
- · Borrar usuarios (solo desactivar) ni reasignar sus atenciones.
- · Editar `seed.py` / `extraer_septiembre.py` (solo reusarlos).
- · Librerías nuevas, abstracciones de un solo uso, capa de config.
- · Tocar el repo legacy `Soporte_Tecnico`.
- · Corregir `tests/test_realtime.py::test_connect_flip_y_disconnect` (fallo pre-existente acoplado al reloj, ver DoD #6).